

TECHNICAL STANDARD OPERATING PROCEDURE

Backups from On-Premises CloudNativePG PostgreSQL to AWS S3

Barman Cloud CNPG-I Plugin | Continuous WAL Archiving | Physical Base Backups | PITR

Document Type	Technical SOP / End-to-End Implementation Guide
Technology	PostgreSQL 18 / Kubernetes / CloudNativePG / Barman Cloud Plugin / AWS S3
Reference Scenario	4-node on-prem Kubernetes cluster; 3-instance CloudNativePG PostgreSQL cluster
Prepared Date	26 August 2026
Classification	Technical Reference / Lab-to-Production Foundation

SOURCE BASIS

This SOP is authored from the supplied reference scenario and is technically cross-checked against official CloudNativePG and official Barman Cloud CNPG-I Plugin documentation. The LinkedIn post is used only as the scenario reference, not as the technical authority.

Prepared for PostgreSQL / Platform Engineering / SRE use

Document Control

Field	Value
Document Title	Backups from On-Premises CloudNativePG PostgreSQL to AWS S3
Document ID	CNPG-S3-BACKUP-SOP-001
Version	1.0
Status	Technical Reference
Prepared Date	26 August 2026
Review Basis	Official CloudNativePG + official Barman Cloud CNPG-I Plugin documentation
Primary Scope	AWS S3 object-store backup, WAL archiving, scheduling, validation, restore and PITR

Revision History

Version	Date	Description
1.0	26-Aug-2026	Initial end-to-end SOP created from the reference scenario and validated against official documentation.

Accuracy and Source Statement

IMPORTANT - ABOUT "100% ACCURACY"

The technical instructions in this document are based exclusively on the cited official CloudNativePG and official Barman Cloud CNPG-I Plugin documentation for the documented features, and were cross-checked during preparation. A literal guarantee of "100% accuracy" is not technically responsible because software releases, APIs, AWS policies, and environment-specific behavior can change. Before production implementation, validate the exact CloudNativePG/plugin versions and manifests against the official documentation for those versions.

The supplied screenshot and LinkedIn post establish the real-world scenario: a four-node on-premises Kubernetes cluster hosts a three-instance PostgreSQL cluster managed by CloudNativePG, while physical backups and continuous WAL archives are sent to AWS S3. The screenshot shows PostgreSQL image ghcr.io/cloudnative-pg/postgresql:18, a healthy three-instance cluster, working WAL archiving, and Barman Cloud Plugin version 0.14.0.

Audience

- PostgreSQL DBAs
- Kubernetes / Platform Engineers
- DevOps and SRE teams
- Backup and Disaster Recovery engineers
- Technical architects evaluating on-premises PostgreSQL backup to AWS S3

Scope

This SOP covers the concepts and implementation flow required to protect a CloudNativePG PostgreSQL cluster running on-premises by sending continuous WAL archives and physical base backups to an AWS S3 bucket using the Barman Cloud CNPG-I Plugin. It also covers backup validation, scheduled backups, restore, PITR, security, retention, monitoring, troubleshooting, and production readiness.

Out of Scope

- Building the Kubernetes cluster itself
- Installing PostgreSQL without CloudNativePG
- Application-level logical backup design with `pg_dump`
- AWS account creation and enterprise IAM governance outside the backup-specific permissions
- Cross-region replica-cluster design beyond recovery fundamentals

Table of Contents

Document Control.....	2
Revision History	2
Accuracy and Source Statement	2
Audience.....	2
Scope.....	2
Out of Scope.....	2
Table of Contents	3
1. Executive Overview.....	6
2. Reference Scenario	6
2.1 Observed Environment.....	6
2.2 Screenshot Validation.....	7
3. Architecture and Data Flow.....	7
3.1 Core Components	7
3.2 Why WAL Archiving Matters.....	8
4. Backup Fundamentals.....	8
4.1 HA Is Not Backup.....	8
4.2 Physical vs Logical Backup	8
4.3 Hot Backup	8
4.4 Recovery Window, RPO and RTO.....	8
5. Prerequisites and Pre-Checks.....	8
5.1 Platform Requirements.....	8
5.2 Kubernetes Health Checks.....	9
5.3 Verify CloudNativePG Version	9
5.4 Verify cert-manager.....	9
5.5 Verify Existing Plugin.....	9
6. AWS S3 Preparation.....	9
6.1 Create and Prepare the Bucket.....	9
6.2 Recommended S3 Controls	9
7. AWS IAM and Authentication	10
7.1 Authentication Choices	10
7.2 Least-Privilege Principle	10
7.3 Create Kubernetes Secret for Static Keys.....	10
8. Install the Barman Cloud Plugin (If Required).....	10
9. Define the AWS S3 ObjectStore.....	10
10. Enable Continuous WAL Archiving	11
10.1 Validate WAL Archiving	12
11. Perform an On-Demand Base Backup	12
11.1 Declarative Backup Resource	12

11.2 Imperative kubectl-cnpg Method	12
12. Configure Scheduled Backups.....	12
13. Backup from Primary vs Standby.....	13
14. Verify Backup and WAL Data	13
14.1 CloudNativePG Status	13
14.2 Kubernetes Backup Resources.....	13
14.3 S3-Side Validation.....	13
15. Retention and S3 Lifecycle	14
16. Full Restore from AWS S3.....	14
17. Point-in-Time Recovery (PITR).....	14
17.1 PITR Validation	15
18. Restore Testing Procedure.....	15
19. Monitoring and Alerting	15
19.1 What to Monitor	15
19.2 Plugin Metrics.....	16
20. Security and Hardening.....	16
21. Network and Firewall Requirements	16
22. Troubleshooting Runbook.....	16
22.1 WAL Archiving Is Not OK.....	16
22.2 Backup Resource Fails.....	17
22.3 Backups Work but Restore Fails	17
23. Common Mistakes.....	17
24. Production Implementation Checklist	17
25. Operational Validation Commands.....	18
26. Recommended Backup Policy Template.....	18
27. Key Takeaways.....	18
28. Official Documentation References	19
28.1 Scenario Reference.....	19
Appendix A - Minimal End-to-End Manifest Set.....	19
Appendix B - Change and Rollback Notes	20

TOC UPDATE

The document uses real Word Heading 1/2/3 styles and contains a TOC field. In Microsoft Word, use Ctrl+A, then F9, and choose "Update entire table" after editing the document.

Quick contents preview (the Word TOC field above remains the authoritative automatic index):

1. Executive Overview
2. Reference Scenario
3. Architecture and Data Flow
4. Backup Fundamentals
5. Prerequisites and Pre-Checks
6. AWS S3 Preparation
7. AWS IAM and Authentication

8. Install the Barman Cloud Plugin (If Required)
 9. Define the AWS S3 ObjectStore
 10. Enable Continuous WAL Archiving
 11. Perform an On-Demand Base Backup
 12. Configure Scheduled Backups
 13. Backup from Primary vs Standby
 14. Verify Backup and WAL Data
 15. Retention and S3 Lifecycle
 16. Full Restore from AWS S3
 17. Point-in-Time Recovery (PITR)
 18. Restore Testing Procedure
 19. Monitoring and Alerting
 20. Security and Hardening
 21. Network and Firewall Requirements
 22. Troubleshooting Runbook
 23. Common Mistakes
 24. Production Implementation Checklist
 25. Operational Validation Commands
 26. Recommended Backup Policy Template
 27. Key Takeaways
 28. Official Documentation References
- Appendix A - Minimal End-to-End Manifest Set
- Appendix B - Change and Rollback Notes

1. Executive Overview

High availability and backup solve different problems. A three-instance CloudNativePG cluster can keep PostgreSQL available during instance or node failures, but replication alone does not protect against accidental DELETE/DROP operations, logical corruption, or complete loss of the Kubernetes environment. A separate backup and WAL archive outside the on-premises failure domain is therefore essential.

The current CloudNativePG backup architecture is plugin-oriented. Starting with CloudNativePG 1.26, native in-tree Barman Cloud object-store integration is deprecated in favor of CNPG-I. The officially supported Barman Cloud Plugin provides online physical backups, continuous WAL archiving, full recovery, and Point-in-Time Recovery (PITR) to supported object stores including AWS S3.

KEY DESIGN

On-premises Kubernetes remains the database execution platform. AWS S3 is the off-cluster object store. CloudNativePG orchestrates the PostgreSQL cluster. The Barman Cloud Plugin performs the object-store backup/WAL operations through sidecars and CNPG-I.

2. Reference Scenario

2.1 Observed Environment

Component	Reference Scenario
Kubernetes	4 nodes: 1 control-plane + 3 worker nodes
Kubernetes Version	v1.30.14 shown in supplied screenshot
Operating System	Ubuntu 24.04.2 LTS shown in supplied screenshot
PostgreSQL Cluster	CloudNativePG cluster named postgres in namespace postgres
PostgreSQL Instances	3: one primary and two synchronous standbys shown
PostgreSQL Image	ghcr.io/cloudnative-pg/postgresql:18 shown
Backup Integration	Barman Cloud CNPG-I Plugin
Plugin Version	0.14.0 shown
Object Store	AWS S3
Backup Mode	Physical base backups + continuous WAL archiving

2.2 Screenshot Validation

```

root@k8smaster:~/cloudnativepg# kubectl get nodes -o wide
NAME        STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
k8smaster   Ready     control-plane   352d   v1.30.14   70.80.90.202   <none>        Ubuntu 24.04.2 LTS   6.14.0-35-generic   containerd://2.2.1
k8snode1    Ready     <none>        352d   v1.30.14   70.80.90.203   <none>        Ubuntu 24.04.2 LTS   6.14.0-35-generic   containerd://2.2.1
k8snode2    Ready     <none>        352d   v1.30.14   70.80.90.204   <none>        Ubuntu 24.04.2 LTS   6.14.0-35-generic   containerd://2.2.1
k8snode3    Ready     <none>        3d4h   v1.30.14   70.80.90.205   <none>        Ubuntu 24.04.2 LTS   6.14.0-35-generic   containerd://2.2.1

root@k8smaster:~/cloudnativepg# kubectl cnpg status postgres -n postgres
Cluster Summary
Name:                postgres/postgres
System ID:            7677975008566079513
PostgreSQL Image:    ghcr.io/cloudnative-pg/postgresql:18
Primary instance:     postgres-1
Primary promotion time: 2026-08-25 14:24:42 +0000 UTC (26h32m59s)
Status:               Cluster in healthy state
Instances:            3
Ready instances:      3
Size:                 272M
Current Write LSN:    0/F000060 (Timeline: 1 - WAL File: 00000001000000000000000F)

Continuous Backup status (Barman Cloud Plugin)
ObjectStore / Server name: postgres-s3/postgres
First Point of Recoverability: 2026-08-26 14:41:30 BST
Last Successful Backup:       2026-08-26 14:41:30 BST
Last Failed Backup:           -
Working WAL archiving:        OK
WALs waiting to be archived:  0
Last Archived WAL:            00000001000000000000000E @ 2026-08-26T16:50:32.893607Z
Last Failed WAL:              00000001000000000000000C @ 2026-08-26T16:44:23.760531Z

Streaming Replication status
Replication Slots Enabled
Name      Sent LSN      Write LSN      Flush LSN      Replay LSN      Write Lag      Flush Lag      Replay Lag      State      Sync State      Sync Priority      Replication Slot
-----
postgres-2 0/F000060    0/F000060    0/F000060    0/F000060    00:00:00    00:00:00    00:00:00    streaming quorum          1              active
postgres-3 0/F000060    0/F000060    0/F000060    0/F000060    00:00:00    00:00:00    00:00:00    streaming quorum          1              active

Instances status
Name      Current LSN      Replication role      Status      QoS      Manager Version      Node
-----
postgres-1 0/F000060      Primary              OK          BestEffort 1.30.0                k8snode3
postgres-2 0/F000060      Standby (sync)       OK          BestEffort 1.30.0                k8snode2
postgres-3 0/F000060      Standby (sync)       OK          BestEffort 1.30.0                k8snode1

Plugins status
Name      Version      Status      Reported Operator Capabilities
-----
barman-cloud.cloudnative-pg.io 0.14.0      N/A          Reconciler Hooks, Lifecycle Service, TYPE_INSTANCE_SIDE CAR_INJECTION

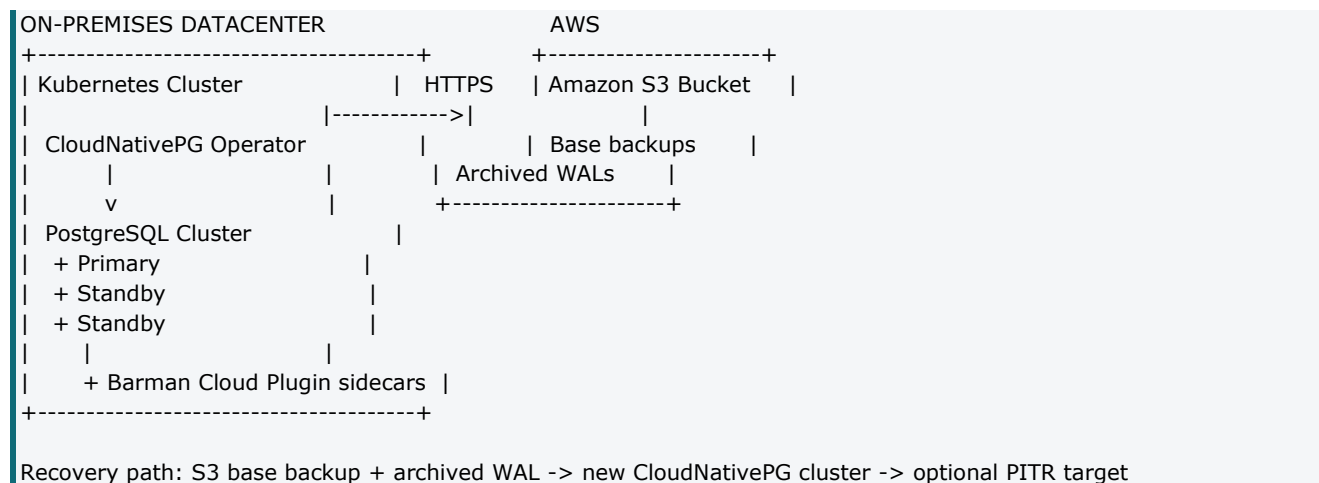
root@k8smaster:~/cloudnativepg#

```

Figure 1 - Supplied CloudNativePG status output showing healthy replication, successful backup status, working WAL archiving, and Barman Cloud Plugin 0.14.0.

The screenshot is evidence of the reference implementation, not a substitute for configuration validation. In production, verify the live Cluster, ObjectStore, Backup/ScheduledBackup resources, Kubernetes Secrets, AWS bucket controls, and restore behavior.

3. Architecture and Data Flow



3.1 Core Components

Component	Responsibility
-----------	----------------

Component	Responsibility
CloudNativePG Operator	Manages PostgreSQL cluster lifecycle and coordinates backup/recovery custom resources.
PostgreSQL	Generates WAL and provides the physical data set used by base backups.
Barman Cloud Plugin	Official CNPG-I object-store integration for physical backup, WAL archive, restore, and PITR.
ObjectStore CR	Defines the target object store, credentials/reference, compression/encryption options, and retention configuration.
Backup CR	Triggers an on-demand physical backup.
ScheduledBackup CR	Creates physical backups on a six-field cron-like schedule.
AWS S3	Durable off-cluster object storage for backup data and WAL archives.
Kubernetes Secret	Stores AWS credentials when static access keys are used.

3.2 Why WAL Archiving Matters

A base backup is a recovery starting point. Continuous WAL archiving preserves the PostgreSQL transaction log after that starting point. Together, the base backup and the required WAL files allow recovery to the latest recoverable state or to a specific point in time. Official CloudNativePG documentation states that object-store based backups require WAL archiving.

4. Backup Fundamentals

4.1 HA Is Not Backup

Replication / HA protects service availability.
 Backup + WAL archive protects recoverability.

DROP TABLE / DELETE / bad application change
 |
 +---> replicated to standbys
 |
 +---> requires backup/PITR to recover earlier state

4.2 Physical vs Logical Backup

CloudNativePG business-continuity backup orchestration is based on PostgreSQL physical backups. Logical `pg_dump` backups can still be useful for object-level export or migration, but the official CloudNativePG backup documentation does not treat `pg_dump` as the business-continuity backup mechanism managed by the operator.

4.3 Hot Backup

The Barman Cloud Plugin supports online physical backups, allowing PostgreSQL to remain available while the base backup is taken. This is the normal object-store backup model for an active production cluster.

4.4 Recovery Window, RPO and RTO

- RPO: how much data loss the business can tolerate. Continuous WAL archiving is central to minimizing the recoverable data gap.
- RTO: how long recovery can take. Base-backup size, network throughput from S3, WAL volume, and WAL replay time all affect recovery duration.
- Recovery Window: retention should preserve at least one usable base backup before the desired recovery point plus all WAL needed to reach points inside the window.

5. Prerequisites and Pre-Checks

5.1 Platform Requirements

- A healthy CloudNativePG-managed PostgreSQL cluster.

- CloudNativePG 1.26 or later for the plugin architecture; current plugin documentation recommends 1.27 or later for improved handling/status reporting.
- Barman Cloud Plugin installed in the same namespace as the CloudNativePG operator, typically cnpg-system.
- cert-manager, which is the recommended method for secure TLS communication between the plugin and operator.
- kubectl access with permissions to create/read the required resources.
- kubectl-cnpg plugin is optional but recommended for status and operational commands.
- Outbound HTTPS connectivity from the Kubernetes workload to the required AWS S3 endpoint.
- An existing S3 bucket and a deliberate IAM/credential strategy.

5.2 Kubernetes Health Checks

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get cluster -n postgres
kubectl cnpg status postgres -n postgres
```

5.3 Verify CloudNativePG Version

```
kubectl get deployment -n cnpg-system cnpg-controller-manager \
-o jsonpath="{.spec.template.spec.containers[*].image}"
```

5.4 Verify cert-manager

```
cmctl check api
```

5.5 Verify Existing Plugin

```
kubectl get pods -n cnpg-system
kubectl get crd objectstores.barmancloud.cnpg.io
kubectl cnpg status postgres -n postgres
```

REFERENCE ENVIRONMENT

The supplied screenshot already reports plugin barman-cloud.cloudnative-pg.io version 0.14.0 and “Working WAL archiving: OK”. Do not reinstall a working plugin simply because this SOP includes installation steps. Treat installation as conditional.

6. AWS S3 Preparation

6.1 Create and Prepare the Bucket

Create the S3 bucket before pointing the PostgreSQL cluster at it. Current Barman Cloud guidance recommends pre-creating and configuring the target bucket rather than depending on backup commands to create it. Choose the AWS Region deliberately and record the bucket name and backup prefix.

```
Example naming only:
Bucket: company-pg-backups-prod
Prefix: postgres/
Destination path used by ObjectStore:
s3://company-pg-backups-prod/postgres/
```

6.2 Recommended S3 Controls

- Block public access.
- Enable S3 Versioning where it fits the organization’s recovery model.
- Use server-side encryption according to enterprise policy.
- Restrict bucket access to the backup identity and required administrators.
- Use an S3 lifecycle policy only after aligning it with Barman retention; do not expire objects earlier than the database recovery policy requires.

- Consider a separate account/region or additional replication controls when S3 is part of a wider disaster-recovery strategy.

S3 LIFECYCLE CAUTION

The official plugin documentation recommends using S3 lifecycle controls as an additional protection layer and keeping lifecycle expiration safely beyond the Barman recovery/retention policy. An aggressive S3 lifecycle can silently destroy the WAL chain needed for PITR.

7. AWS IAM and Authentication

7.1 Authentication Choices

For an on-premises Kubernetes cluster, the simplest lab pattern is an AWS access key and secret key stored in a Kubernetes Secret. In production, use the organization's approved short-lived or externally managed AWS credential mechanism where possible. The official plugin documentation also supports IAM Role for Service Account (IRSA) for EKS; that mechanism is specific to Amazon EKS and is not directly the normal on-premises pattern.

7.2 Least-Privilege Principle

Grant only the S3 actions required for the selected Barman Cloud operations on the intended bucket/prefix. Do not use an AWS administrator credential. IAM details vary by security policy and Barman feature set, so validate the final policy against the current official plugin/AWS requirements before production deployment.

7.3 Create Kubernetes Secret for Static Keys

```
kubectl -n postgres create secret generic aws-creds \
  --from-literal=ACCESS_KEY_ID='<AWS_ACCESS_KEY_ID>' \
  --from-literal=ACCESS_SECRET_KEY='<AWS_SECRET_ACCESS_KEY>'

kubectl -n postgres get secret aws-creds
```

SECRET HANDLING

Never commit real AWS access keys into Git, YAML examples, tickets, screenshots, or documentation. Use placeholders in manifests and a controlled secret-delivery process.

8. Install the Barman Cloud Plugin (If Required)

Plugin version 0.14.0 is shown in the supplied environment. For a new environment, use the manifest for the exact supported release selected by your change process. Official 0.14.0 installation documentation uses the following manifest.

```
kubectl apply -f \
  https://github.com/cloudnative-pg/plugin-barman-cloud/releases/download/v0.14.0/manifest.yaml
```

The plugin must be installed in the same namespace as the CloudNativePG operator. Verify the resulting deployment/pods and the ObjectStore CRD before configuring the database cluster.

```
kubectl get pods -n cnpg-system
kubectl get crd objectstores.barmancloud.cnpg.io
```

9. Define the AWS S3 ObjectStore

The ObjectStore custom resource is the connection definition between CloudNativePG/Barman Cloud Plugin and the target object storage. The example below uses static AWS credentials because the reference cluster is on-premises.

```

apiVersion: barmancloud.cnpg.io/v1
kind: ObjectStore
metadata:
  name: postgres-s3
  namespace: postgres
spec:
  configuration:
    destinationPath: "s3://company-pg-backups-prod/postgres/"
    s3Credentials:
      accessKeyId:
        name: aws-creds
        key: ACCESS_KEY_ID
      secretAccessKey:
        name: aws-creds
        key: ACCESS_SECRET_KEY
    wal:
      compression: gzip
      retentionPolicy: "30d"

```

CONFIGURATION NOTE

Replace the bucket name, prefix, retention period, and optional compression/encryption settings with values approved for your environment. In the plugin architecture, retention belongs to the ObjectStore resource.

```

kubectl apply -f objectstore-postgres-s3.yaml
kubectl get objectstore -n postgres
kubectl describe objectstore postgres-s3 -n postgres

```

10. Enable Continuous WAL Archiving

Reference the ObjectStore from the CloudNativePG Cluster through the plugin stanza and set `isWALArchiver` to true. This delegates WAL archiving to the Barman Cloud Plugin.

```

apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres
  namespace: postgres
spec:
  instances: 3
  # Existing storage, resources, PostgreSQL settings, etc. remain here.

  plugins:
    - name: barman-cloud.cloudnative-pg.io
      isWALArchiver: true
      parameters:
        barmanObjectName: postgres-s3

```

For an existing cluster, edit only the required portion of the real manifest and apply it through the normal GitOps/change-control path. A plugin migration or configuration change can trigger reconciliation/rollout, so review the exact diff before applying it.

```

kubectl apply -f postgres-cluster.yaml
kubectl cnpg status postgres -n postgres

```

10.1 Validate WAL Archiving

```
kubectl cnpg status postgres -n postgres

# PostgreSQL-side validation
kubectl exec -n postgres -it <PRIMARY_POD> -- psql -c \
"SELECT archived_count, failed_count, last_archived_wal, last_archived_time,
    last_failed_wal, last_failed_time
FROM pg_stat_archiver;"
```

A healthy status should show working WAL archiving and no growing failure count. The supplied screenshot shows “Working WAL archiving: OK”, zero WALs waiting to be archived, and a recent last archived WAL.

11. Perform an On-Demand Base Backup

11.1 Declarative Backup Resource

```
apiVersion: postgresql.cnpg.io/v1
kind: Backup
metadata:
  name: postgres-manual-001
  namespace: postgres
spec:
  cluster:
    name: postgres
  method: plugin
  pluginConfiguration:
    name: barman-cloud.cloudnative-pg.io

kubectl apply -f backup-postgres-manual.yaml
kubectl get backup -n postgres -w
kubectl describe backup postgres-manual-001 -n postgres
```

11.2 Imperative kubectl-cnpg Method

```
kubectl cnpg backup -n postgres postgres \
--method=plugin \
--plugin-name=barman-cloud.cloudnative-pg.io
```

The plugin documentation supports both declarative Backup resources and the kubectl-cnpg backup command. For repeatable production operations, declarative resources and Git-controlled configuration are generally easier to audit.

12. Configure Scheduled Backups

CloudNativePG recommends ScheduledBackup for a reliable recurring strategy. The schedule uses six fields, including seconds, unlike standard five-field Linux crontab syntax.

```
apiVersion: postgresql.cnpg.io/v1
kind: ScheduledBackup
metadata:
  name: postgres-daily
  namespace: postgres
spec:
  schedule: "0 0 0 * * *" # Every day at 00:00:00
  backupOwnerReference: self
  cluster:
    name: postgres
  method: plugin
  pluginConfiguration:
    name: barman-cloud.cloudnative-pg.io
```

```
kubectl apply -f scheduledbackup-postgres.yaml
kubectl get scheduledbackup -n postgres
kubectl describe scheduledbackup postgres-daily -n postgres
kubectl get backup -n postgres
```

SCHEDULE FORMAT

CloudNativePG ScheduledBackup uses a six-field cron expression: seconds, minutes, hours, day-of-month, month, day-of-week. "0 0 0 * * *" means midnight every day.

13. Backup from Primary vs Standby

A physical base backup reads the database data set and can create I/O load. CloudNativePG supports taking backups from a standby to reduce impact on the primary. The cluster-wide backup target defaults to prefer-standby in the native API behavior documented by CloudNativePG, and a Backup/ScheduledBackup can override the target where supported.

```
# Example per-backup override concept
spec:
  cluster:
    name: postgres
  target: primary
  method: plugin
  pluginConfiguration:
    name: barman-cloud.cloudnative-pg.io
```

DESIGN DECISION

Prefer a healthy standby when backup I/O should be offloaded from the primary, but validate replica performance and recovery behavior. Forcing backups to the primary is appropriate only when that trade-off is intentional.

14. Verify Backup and WAL Data

14.1 CloudNativePG Status

```
kubectl cnpg status postgres -n postgres
```

For the reference environment, the status output exposes Continuous Backup status, First Point of Recoverability, Last Successful Backup, Last Failed Backup, Working WAL archiving, WALs waiting to be archived, and the most recent archived/failed WAL information.

14.2 Kubernetes Backup Resources

```
kubectl get backup -n postgres
kubectl get scheduledbackup -n postgres
kubectl describe backup <BACKUP_NAME> -n postgres
```

14.3 S3-Side Validation

Validate that the intended bucket/prefix contains the Barman Cloud backup catalog structure and that object timestamps continue to advance. The plugin follows Barman Cloud conventions for base backups and WAL storage. Do not manually rename or reorganize backup/WAL objects.

```
# Example only, if AWS CLI is installed and authorized:
aws s3 ls s3://company-pg-backups-prod/postgres/ --recursive
```

BACKUP SUCCESS CRITERIA

A successful Kubernetes Backup object is necessary but not sufficient. Confirm continuous WAL archiving, object-store visibility, retention behavior, and - most importantly - a successful restore/PITR test.

15. Retention and S3 Lifecycle

The Barman Cloud Plugin ObjectStore can define a recovery-window retention policy. Retention must be designed together with base-backup frequency and WAL archiving so the full WAL chain required by the recovery window remains available.

```
spec:
  retentionPolicy: "30d"
```

S3 lifecycle is separate from Barman retention. Use lifecycle controls as an additional storage-governance mechanism, but configure them so they do not delete current or non-current objects earlier than the recovery policy allows. Versioning can provide additional protection against accidental deletion when aligned with security and cost requirements.

16. Full Restore from AWS S3

CloudNativePG recovery is not an in-place overwrite of the existing cluster. Recovery bootstraps a new PostgreSQL Cluster from an available physical backup and replays WAL from the object store. This design lets the original cluster remain available for investigation or controlled cutover.

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres-restore
  namespace: postgres
spec:
  instances: 3

  bootstrap:
    recovery:
      source: source

  externalClusters:
    - name: source
      plugin:
        name: barman-cloud.cloudnative-pg.io
        parameters:
          barmanObjectName: postgres-s3
          serverName: postgres

  storage:
    size: <MATCH_OR_EXCEED_REQUIRED_SIZE>

kubect! apply -f postgres-restore.yaml
kubect! get cluster postgres-restore -n postgres -w
kubect! get pods -n postgres -w
kubect! cnpg status postgres-restore -n postgres
```

RESTORED CLUSTER ARCHIVING

The recovery ObjectStore is used to read the source backup/WAL. If the restored cluster must begin writing its own new backups/WAL archive, configure its own plugin/ObjectStore destination deliberately. Avoid accidental namespace/server-name collisions in object storage.

17. Point-in-Time Recovery (PITR)

PITR restores a base backup and replays archived WAL only up to a chosen recovery target. A valid WAL archive is mandatory. This is the normal recovery approach for incidents such as accidental data deletion when the required recovery point is known.

```

apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres-pitr
  namespace: postgres
spec:
  instances: 3
  storage:
    size: <REQUIRED_SIZE>

  bootstrap:
    recovery:
      source: source
      recoveryTarget:
        targetTime: "2026-08-26 14:40:00+00"

  externalClusters:
    - name: source
  plugin:
    name: barman-cloud.cloudnative-pg.io
    parameters:
      barmanObjectName: postgres-s3
      serverName: postgres

```

17.1 PITR Validation

1. Confirm the requested timestamp is inside the recoverable window.
2. Recover to a new cluster name.
3. Wait until the new cluster becomes primary and accepts writes.
4. Validate business data, schema, extensions, roles/secrets, and application consistency.
5. Record actual restore duration and compare it with RTO.
6. Only then plan application cutover or data extraction.

18. Restore Testing Procedure

A backup strategy is incomplete until restore is tested. Perform restore testing on a planned schedule and after major changes to PostgreSQL, CloudNativePG, the Barman plugin, credentials, storage, or S3 policy.

7. Choose a known successful backup and record its timestamp.
8. Create an isolated restore namespace or unique cluster name.
9. Restore from S3 to a new CloudNativePG cluster.
10. Validate PostgreSQL starts and replicas become healthy.
11. Check database list, critical table counts, extensions, roles, and application smoke tests.
12. Perform a PITR test to a known transaction/time where possible.
13. Measure download + WAL replay + cluster readiness time.
14. Document RPO/RTO evidence.
15. Delete the temporary restore cluster only after validation is signed off.

19. Monitoring and Alerting

19.1 What to Monitor

- Last successful backup timestamp
- Last failed backup timestamp
- First point of recoverability
- WAL archiving status
- WALs waiting to be archived
- Last archived WAL and time

- Last failed WAL and time
- Backup resource phase/failure
- Barman Cloud Plugin health and sidecar errors
- S3 access errors, throttling, DNS/network failures
- Kubernetes pod restarts and resource pressure
- PostgreSQL replication health
- Bucket storage growth and lifecycle/retention behavior

19.2 Plugin Metrics

When using the plugin architecture, monitor the plugin-specific backup metrics rather than relying only on older in-core Barman metric names. Official migration documentation identifies plugin metrics such as

`barman_cloud_cloudnative_pg_io_last_failed_backup_timestamp`,
`barman_cloud_cloudnative_pg_io_last_available_backup_timestamp`, and
`barman_cloud_cloudnative_pg_io_first_recoverability_point`.

20. Security and Hardening

- Use least-privilege AWS access scoped to the required bucket/prefix.
- Do not store AWS secrets in source control.
- Restrict Kubernetes RBAC access to Secrets and ObjectStore resources.
- Use encrypted transport to AWS S3.
- Apply S3 Block Public Access.
- Use server-side encryption based on enterprise requirements.
- Rotate static credentials and test the rotation procedure.
- Separate production and non-production backup destinations.
- Protect deletion privileges and consider S3 Versioning / governance controls.
- Audit access to backup objects and Kubernetes Secrets.
- Ensure S3 lifecycle cannot violate the database retention/recovery window.
- Treat backup data as production-sensitive data because it contains the full database.

21. Network and Firewall Requirements

The on-premises Kubernetes nodes/pods performing backup and WAL archive operations require outbound connectivity to AWS S3 over HTTPS and working DNS. Exact endpoint routing depends on the AWS Region, corporate proxy, firewall, and DNS design. Test connectivity from the actual workload path rather than only from an administrator workstation.

```
# Generic diagnostics from an authorized troubleshooting pod/container:
nslookup <regional-s3-endpoint>
curl -I https://<regional-s3-endpoint>

# Then validate through CloudNativePG/Barman status and logs rather than
# treating a successful curl alone as proof that AWS authentication works.
```

22. Troubleshooting Runbook

22.1 WAL Archiving Is Not OK

16. Check `kubectcl cnpg status` and `pg_stat_archiver`.
17. Check primary pod and Barman sidecar logs.
18. Verify ObjectStore name/namespace and plugin reference.
19. Verify Kubernetes Secret key names exactly match ObjectStore references.
20. Check AWS credential validity and IAM permissions.
21. Check DNS, firewall, proxy, TLS, and time synchronization.

22. Check bucket name, Region, prefix, and S3 policy.
23. Confirm S3 lifecycle or manual deletion has not broken the archive.

22.2 Backup Resource Fails

```
kubectl get backup -n postgres
kubectl describe backup <BACKUP_NAME> -n postgres
kubectl get events -n postgres --sort-by=.lastTimestamp
kubectl cnpg status postgres -n postgres
```

Correlate the Backup status with PostgreSQL pod logs, Barman plugin sidecar logs, S3 permissions, and network connectivity. Do not repeatedly retry a failed backup without understanding the error, especially when the failure involves an existing archive or server-name safety check.

22.3 Backups Work but Restore Fails

- Confirm the backup catalog is visible and a valid base backup exists.
- Confirm all WAL required for the requested target is present.
- Confirm recovery ObjectStore/serverName points to the source cluster backup identity.
- Check restored storage size and storage class.
- Check PostgreSQL major-version/extension compatibility for the recovery design.
- Check Kubernetes Secrets needed by applications; CloudNativePG recovery does not automatically make external Kubernetes secret management disappear.
- Inspect recovery pod/job and plugin logs.

23. Common Mistakes

Mistake	Impact / Correction
Assuming three replicas are backups	Replication reproduces logical mistakes. Keep independent base backups and WAL archive.
Using old .spec.backup.barmanObjectStore examples for a new design	Native Barman object-store integration is deprecated from CNPG 1.26; use the official Barman Cloud Plugin architecture.
Using five-field cron syntax	ScheduledBackup uses six fields including seconds.
Hard-coding AWS keys in YAML/Git	Use Kubernetes Secret or approved external credential delivery.
Giving AWS AdministratorAccess	Use least privilege for the specific backup bucket/prefix.
S3 lifecycle shorter than recovery retention	Can remove WAL/base backups needed for PITR.
Never testing restore	A successful backup job does not prove recoverability.
Restoring over the running cluster	CloudNativePG recovery bootstraps a new cluster.
Reusing object-store identity carelessly	Can cause archive collision/safety-check failures; isolate cluster backup identities.
Ignoring plugin/version compatibility	Validate the exact CloudNativePG and plugin versions before change.

24. Production Implementation Checklist

- ☐ CloudNativePG and Barman Cloud Plugin versions are compatible and supported.
- ☐ cert-manager/plugin health is verified.
- ☐ S3 bucket exists in the approved Region.
- ☐ S3 Block Public Access is enabled.
- ☐ Encryption and Versioning decisions are documented.
- ☐ AWS identity uses least privilege.
- ☐ Credentials are delivered securely and are rotatable.
- ☐ ObjectStore is validated in the PostgreSQL namespace.
- ☐ Continuous WAL archiving reports healthy.

- ☐ A manual base backup completes successfully.
- ☐ ScheduledBackup is configured with correct six-field schedule.
- ☐ Retention policy is documented.
- ☐ S3 lifecycle is longer/safer than the recovery requirement.
- ☐ Monitoring alerts exist for backup failure and WAL archive failure.
- ☐ A full restore to a new cluster has succeeded.
- ☐ A PITR exercise has succeeded.
- ☐ Actual RPO and RTO have been measured.
- ☐ Backup and restore runbooks are owned and reviewed.

25. Operational Validation Commands

```
# Cluster and backup health
kubectl cnpg status postgres -n postgres
kubectl get cluster -n postgres
kubectl get backup -n postgres
kubectl get scheduledbackup -n postgres
kubectl get objectstore -n postgres

# Pods and events
kubectl get pods -n postgres -o wide
kubectl get events -n postgres --sort-by=.lastTimestamp

# PostgreSQL WAL archiver
kubectl exec -n postgres -it <PRIMARY_POD> -- psql -c \
"SELECT * FROM pg_stat_archiver;"

# Replication
kubectl exec -n postgres -it <PRIMARY_POD> -- psql -c \
"SELECT application_name, state, sync_state, replay_lsn FROM pg_stat_replication;"
```

26. Recommended Backup Policy Template

Policy Item	Example / Decision to Record
Backup Method	Barman Cloud CNPG-I Plugin to AWS S3
WAL Archiving	Continuous
Base Backup Frequency	Daily / weekly according to measured RTO and policy
Backup Target	prefer-standby or primary based on workload design
Retention Window	Example: 30d; set from business requirement
S3 Versioning	Enabled/Disabled with justification
S3 Encryption	Enterprise-approved server-side encryption
Restore Test	Monthly/quarterly and after major changes
PITR Test	Regular controlled exercise
RPO	Business-approved target
RTO	Business-approved target, validated by test
Alert Ownership	DBA / SRE on-call group

27. Key Takeaways

- CloudNativePG HA and AWS S3 backup are complementary, not interchangeable.
- For current CloudNativePG deployments, the Barman Cloud CNPG-I Plugin is the recommended official object-store path; the older in-tree Barman object-store integration is deprecated from 1.26.
- The ObjectStore CR defines the S3 destination and backup/WAL behavior; the Cluster plugin stanza enables continuous WAL archiving.

- Backup and ScheduledBackup resources create physical base backups using method: plugin.
- PITR requires both a usable base backup and an unbroken WAL archive.
- Recovery creates a new CloudNativePG cluster rather than overwriting the running cluster.
- Retention, S3 lifecycle, IAM, network access, monitoring, and restore testing are all part of the backup design.
- The strongest proof of a backup is a successful, measured restore.

28. Official Documentation References

The technical implementation in this SOP was cross-checked against the following official documentation. These links should be revalidated against the exact versions used in production:

- [Barman Cloud Plugin - Introduction](#)
- [Barman Cloud Plugin - Installation \(0.14.0 current page\)](#)
- [Barman Cloud Plugin - Object Store Providers / AWS S3](#)
- [Barman Cloud Plugin - Usage](#)
- [Barman Cloud Plugin - Migration from Built-in Backup](#)
- [CloudNativePG - Backup](#)
- [CloudNativePG - Recovery](#)
- [CloudNativePG - Operator Capability Levels](#)

28.1 Scenario Reference

Reference LinkedIn scenario supplied by the user: [Backups From On-Prem CloudNativePG PostgreSQL Cluster To AWS S3 Buckets](#)

AUTHORITY NOTE

The LinkedIn post and supplied screenshot are scenario evidence only. Configuration and operational claims in this SOP use official CloudNativePG/Barman Cloud Plugin documentation as the technical authority.

Appendix A - Minimal End-to-End Manifest Set

The following compact set shows the relationship between the Secret, ObjectStore, Cluster plugin configuration, and ScheduledBackup. Replace all placeholders and merge the Cluster fragment into the real cluster manifest rather than replacing unrelated production settings.

```
# 1) Kubernetes Secret (create imperatively or through your secret manager)
# kubectl -n postgres create secret generic aws-creds \
# --from-literal=ACCESS_KEY_ID='<ID>' \
# --from-literal=ACCESS_SECRET_KEY='<SECRET>'

---
apiVersion: barmancloud.cnpg.io/v1
kind: ObjectStore
metadata:
  name: postgres-s3
  namespace: postgres
spec:
  configuration:
    destinationPath: "s3://<BUCKET>/<PREFIX>/"
    s3Credentials:
      accessKeyId:
        name: aws-creds
        key: ACCESS_KEY_ID
      secretAccessKey:
        name: aws-creds
        key: ACCESS_SECRET_KEY
  wal:
    compression: gzip
    retentionPolicy: "30d"

---
# Merge this plugins section into the existing CloudNativePG Cluster.
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres
  namespace: postgres
spec:
  instances: 3
  plugins:
    - name: barman-cloud.cloudnative-pg.io
      isWALArchiver: true
      parameters:
        barmanObjectName: postgres-s3
  storage:
    size: <EXISTING_OR_REQUIRED_SIZE>

---
apiVersion: postgresql.cnpg.io/v1
kind: ScheduledBackup
metadata:
  name: postgres-daily
  namespace: postgres
spec:
  schedule: "0 0 0 * * *"
  backupOwnerReference: self
  cluster:
    name: postgres
  method: plugin
  pluginConfiguration:
    name: barman-cloud.cloudnative-pg.io
```

Appendix B - Change and Rollback Notes

Before enabling the plugin on an existing production cluster, capture the current Cluster manifest, current backup configuration, CloudNativePG/operator version, plugin version, S3 policy, and credential state. Apply the change

through the normal version-controlled process. If migrating from deprecated in-tree Barman Cloud, follow the official migration sequence rather than running both designs casually in parallel.

24. Pre-change: verify current backup and WAL health.
25. Pre-change: create a fresh known-good backup if the existing mechanism permits it.
26. Apply ObjectStore and validate it.
27. Apply the Cluster plugin change atomically as documented for migration.
28. Update ScheduledBackup resources to method: plugin.
29. Verify WAL archiving immediately.
30. Run a manual plugin backup.
31. Verify S3 objects and metrics.
32. Perform a restore test before declaring the change complete.